

## Implementation of Transaction Properties (ACID) in Freelance Marketplace System

### Aim

To implement and demonstrate the ACID properties (Atomicity, Consistency, Isolation, and Durability) using SQL transactions in a Freelance Marketplace Database.

### Scenario

A client accepts a freelancer's application for a project.

The transaction involves:

1. Updating the project status to Assigned
2. Creating a contract between the client and freelancer
3. Changing the application status to Accepted

These operations must either all succeed or all fail.

#### 1. Atomicity

Atomicity ensures that either all operations of the transaction are completed successfully or none of them are applied.

### Database Tables

CLIENTS(ClientID PRIMARY KEY, ClientName, ClientEmail)

FREELANCERS(FreelancerID PRIMARY KEY, FreelancerName, Skills, Experience)

PROJECTS(ProjectID PRIMARY KEY, ClientID, ProjectTitle, Budget, Status, Foreign Key: ClientID references CLIENTS)

APPLICATIONS(ApplicationID PRIMARY KEY, ProjectID, FreelancerID, BidAmount, Status, Foreign Keys: ProjectID references PROJECTS and FreelancerID references FREELANCERS)

CONTRACTS(ContractID PRIMARY KEY, ProjectID, FreelancerID, StartDate, ContractAmount, Foreign Keys: ProjectID references PROJECTS and FreelancerID references FREELANCERS)

#### 1. Atomicity

Atomicity ensures that all operations in a transaction are completed successfully or none of them are applied.

```

[mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Projects
  -> SET Status='Assigned'
  -> WHERE ProjectID=101;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> INSERT INTO Contracts
  -> VALUES
  -> (6,101,201,CURDATE(),45000);
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE Applications
  -> Set Status='Accepted'
  -> WHERE ApplicationID=301;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

```

## Verification

```

[mysql> SELECT * FROM Projects;
+-----+-----+-----+-----+-----+
| ProjectID | ClientID | ProjectTitle | Budget | Status |
+-----+-----+-----+-----+-----+
| 101 | 1 | E-commerce Website | 50000.00 | Assigned |
| 102 | 2 | Mobile App Development | 70000.00 | Open |
| 103 | 3 | AI Chatbot | 60000.00 | Open |
| 104 | 1 | Portfolio Website | 30000.00 | Open |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

[mysql> SELECT * FROM Contracts;
+-----+-----+-----+-----+-----+
| ContractID | ProjectID | FreelancerID | StartDate | ContractAmount |
+-----+-----+-----+-----+-----+
| 1 | 101 | 201 | 2026-07-10 | 45000.00 |
| 6 | 101 | 201 | 2026-07-10 | 45000.00 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

[mysql> SELECT * FROM Applications;
+-----+-----+-----+-----+-----+
| ApplicationID | ProjectID | FreelancerID | BidAmount | Status |
+-----+-----+-----+-----+-----+
| 301 | 101 | 201 | 45000.00 | Accepted |
| 302 | 102 | 202 | 65000.00 | Pending |
| 303 | 103 | 203 | 55000.00 | Pending |
| 304 | 104 | 201 | 28000.00 | Pending |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

## 2. Consistency

Consistency ensures that the database remains in a valid state.

### Valid Transaction

```
mysql> INSERT INTO Contracts
-> VALUES
[    -> (3,103,203,CURDATE(),55000);
Query OK, 1 row affected (0.00 sec)
```

Invalid Transaction

```
mysql> INSERT INTO Contracts
-> VALUES
[    -> (4,500,203,CURDATE(),30000);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`freelance_marketplace`.`contracts`,
CONSTRAINT `contracts_ibfk_1` FOREIGN KEY (`ProjectID`) REFERENCES `projects` (`ProjectID`))
```

### 3. Isolation

Isolation ensures that concurrent transactions do not interfere with each

other. Query A

```
[mysql> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Projects
-> SET Status='Assigned'
[    -> WHERE ProjectID=104;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0
```

Query B

```
mysql> SELECT Status
-> FROM Projects
[    -> WHERE ProjectID=104;
+-----+
| Status |
+-----+
| Assigned |
+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

### 4. Durability

Durability ensures that committed changes are permanently stored.

```
[mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

[mysql> INSERT INTO Contracts VALUES (31,104,203,CURDATE(),28000);
Query OK, 1 row affected (0.00 sec)

[mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

#### Verification

```
[mysql> SELECT * FROM Contracts;
```

| ContractID | ProjectID | FreelancerID | StartDate  | ContractAmount |
|------------|-----------|--------------|------------|----------------|
| 1          | 101       | 201          | 2026-07-10 | 45000.00       |
| 3          | 103       | 203          | 2026-07-10 | 55000.00       |
| 6          | 101       | 201          | 2026-07-10 | 45000.00       |
| 31         | 104       | 203          | 2026-07-10 | 28000.00       |

```
4 rows in set (0.00 sec)
```

#### Result

The ACID properties—Atomicity, Consistency, Isolation, and Durability—were successfully implemented and verified in the Freelance Marketplace database using SQL transaction commands.